

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ
по лабораторной работе № 6
по курсу «Разработка кроссплатформенных приложений»
Вариант 3

Выполнили
студенты группы 22ВОЭ1
Хворов Ф. М.
Адов А. А.

Приняли
Юрова О. В.

Пенза 2025

Цель работы

Научиться создавать клиент-серверные приложения с использованием стандартных классов Java.

Задание

Модифицировать приложение из предыдущей лабораторной работы, реализовав клиент-серверную архитектуру, обеспечивающую распределенное вычисление определенного интеграла на нескольких вычислительных узлах (клиентах) при этом каждый узел использует несколько нитей, как в предыдущей работе. Сервер не занимается вычислениями, а лишь реализует взаимодействие с пользователем и агрегацию результатов вычислений от клиентов.

Исходный код программы

```
private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {  
    DefaultTableModel tModel = (DefaultTableModel) jTable1.getModel();  
  
    for (int row = 0; row < tModel.getRowCount(); row++) {  
        final int finalRow = row;  
        double a = ((Number) tModel.getValueAt(row, 0)).doubleValue();  
        double b = ((Number) tModel.getValueAt(row, 1)).doubleValue();  
        double step = ((Number) tModel.getValueAt(row, 2)).doubleValue();  
  
        // Разбиваем интервал на 3 части  
        double interval = (b - a) / 3;  
        final double[] totalResult = {0.0}; // Массив вместо AtomicReference  
  
        // Создаем 3 потока  
        Thread[] threads = new Thread[3];  
        for (int i = 0; i < 3; i++) {  
            final double start = a + i * interval;
```

```

        final double end = (i == 2) ? b : start + interval;
        threads[i] = new Thread(() -> {
            double partialSum = 0;
            for (double x = start; x < end; x += step) {
                partialSum += Math.cos(x + step / 2) * step;
            }
            synchronized (totalResult) {
                totalResult[0] += partialSum; // Безопасное обновление
            }
        });
        threads[i].start();
    }

    // Ждем завершения потоков
    for (Thread thread : threads) {
        try {
            thread.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    // Обновляем GUI и отправляем на сервер
    SwingUtilities.invokeLater(() -> {
        tModel.setValueAt(totalResult[0], finalRow, 3);
        integralsList.get(finalRow).setResult(totalResult[0]);
        sendResultToServer(a, b, step, totalResult[0]);
    });
}

private void sendResultToServer(double a, double b, double step, double result) {
    try {
        String data = String.format("a=%.2f, b=%.2f, step=%.4f, result=%.6f", a, b, step, result);
        byte[] buffer = data.getBytes();
        DatagramPacket packet = new DatagramPacket(buffer, buffer.length, serverAddress,
SERVER_PORT);
        socket.send(packet);
    } catch (Exception e) {
        System.err.println("Ошибка отправки: " + e.getMessage());
    }
}

```

Выполнение программы

Нижняя граница Верхняя граница Шаг интегрирования

Нижняя граница	Верхняя граница	Шаг	Результат
1.0	100.0	0.5	-1.36197964452...
1.0	2.0	0.5	-0.01811496013...

Добавить

Удалить

Рассчитать

Run:

Server Zapushen i ozhidaet rezultatov

Result from /127.0.0.1: a=1,00, b=100,00, step=0,5000, result=-1,361980

Result from /127.0.0.1: a=1,00, b=2,00, step=0,5000, result=-0,018115

Рисунок 1 — Вычисление и работа сервера

Ход работы

```
import java.net.*;

public class Server {
    public static void main(String[] args) throws Exception {
        DatagramSocket socket = new DatagramSocket(9876);
        System.out.println("Server Zapushen i ozhidaet rezultatov");

        while (true) {
            byte[] buffer = new byte[1024];
            DatagramPacket packet = new DatagramPacket(buffer, buffer.length);
            socket.receive(packet);

            String message = new String(packet.getData(), 0, packet.getLength());

            if (message.equals("CLIENT_CONNECTED")) {

                String response = "SERVER_ACK";
                byte[] responseData = response.getBytes();
                DatagramPacket responsePacket = new DatagramPacket(
                    responseData, responseData.length,
                    packet.getAddress(), packet.getPort()
                );
                socket.send(responsePacket);
            } else {

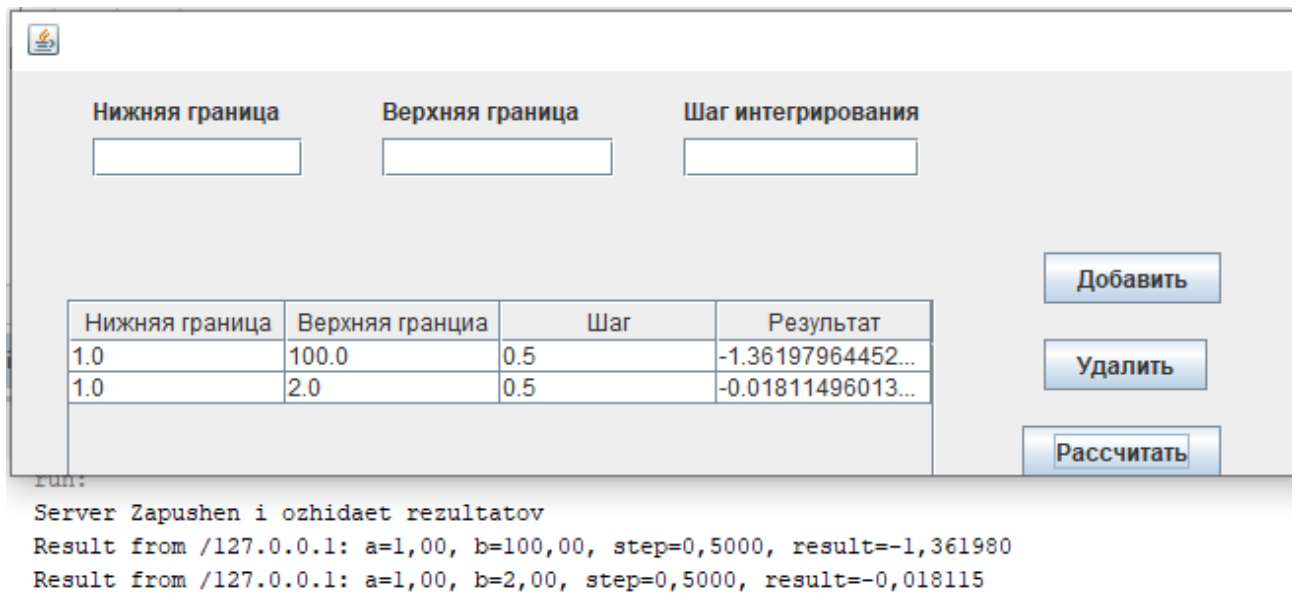
                System.out.println("Result from " + packet.getAddress() + ": " + message);
            }
        }
    }
}
```

Пояснение к тексту программы(ОСНОВНЫЕ ВЫЧИСЛЕНИЯ)

Строки 5: Задается сокет.

Строки 6 — 24: Происходит работа сервера, включающая в себя получение пакета информации, и его обработка.

Результат выполнения программы



Нижняя граница Верхняя граница Шаг интегрирования

Нижняя граница	Верхняя граница	Шаг	Результат
1.0	100.0	0.5	-1.36197964452...
1.0	2.0	0.5	-0.01811496013...

Добавить

Удалить

Рассчитать

run:
Server Zapushen i ozhidaet rezultatov
Result from /127.0.0.1: a=1,00, b=100,00, step=0,5000, result=-1,361980
Result from /127.0.0.1: a=1,00, b=2,00, step=0,5000, result=-0,018115

Рисунок 2 — Результат

Вывод

Изучена работа с серверами и создано клиент-серверное приложение.